VIDEO GAMES SALES PREDICTION

BACHELOR OF TECHNOLOGY
IN

COMPUTER SCIENCE AND ENGINEERING

BY

K. Varun	**23501A0569**
K. Jaya Prakash	**23501A0578**
K. Jahnavi	**23501A0567**
M. Poshanya Guptha	**23501A05A6**
K. Yasaswini	**23501A0588**

PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY

(Permanently affiliated to JNTU Kakinada, Approved by AICTE)

(An NBA & NAAC A+ accredited and ISO 21001:2018 Certified Institution)

Kanuru, Vijayawada – 520007

(2025-26)

PRASAD V POTLURI

SIDDHARTHA INSTITUTE OF TECHNOLOGY

(Permanently affiliated to JNTU Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 21001:2018 certified institution)

Kanuru, Vijayawada – 520007

CERTIFICATE

This is to certify that the project report title **"Video Games Sales Prediction"** is the bonafied work of **K. Varun (23501A0569), K. Jaya Prakash (23501A0578), K. Jahnavi (23501A0567), M. Poshanya Guptha (23501A05A6), K. Yasaswini (23501A0588)** in partial fulfilment of completing the Academic project in Data Warehousing & Data Mining during the academic year 2025-26.

Signature of the course coordinator **Signature of the HOD**

I N D E X

1. Abstract

This machine learning project aimed to develop and evaluate predictive models for global video game sales to provide publishers, developers, and retailers with actionable insights, ultimately helping to reduce business risks in the highly competitive entertainment sector. The project utilizes a dataset of 16,598 video game records scraped from vgchartz.com, which includes attributes like Platform, Genre, Critic_Score, and User_Score. The target variable for prediction is Global_Sales, measured in millions.

A crucial first step was Data Preprocessing to handle the raw data's inconsistencies and missing values. Missing values in numerical features like Critic_Score and User_Score were addressed using imputation with the mean. Missing categorical entries for features like Publisher and Rating were filled with the mode (most frequent value). Irrelevant attributes such as Name and Rank were excluded. Feature Engineering was performed by deriving the Age of the Game from the release Year to capture temporal sales trends. To prepare the data for regression models, Label Encoding was applied to categorical features like Rating, Platform, and Genre. Furthermore, to ensure uniform scaling for distance-based algorithms, Z-Score Normalization was applied to numerical columns. Most importantly, a log transformation (log1p) was applied to the highly skewed Global_Sales target variable to normalize the distribution and improve model performance, especially for games with modest sales.

The preprocessed data was then split into an 80:20 train-test ratio. Five different regression algorithms were trained and evaluated: K-Nearest Neighbour Regressor, Linear Regression, Support Vector Regressor (SVR), Random Forest Regressor, and XG Boost Regressor. Hyperparameter Tuning was performed using Grid Search with 5-fold cross-validation to optimize model performance. Model performance was assessed using two primary metrics: Root Mean Squared Error (RMSE) and the R^2 Score (Coefficient of Determination).

The evaluation results clearly showed the XG Boost Regressor as the superior model. It achieved the lowest RMSE (0.2885) and the highest R^2 score (0.6528), indicating that it could explain approximately 65.3% of the variance in the log-transformed global sales. This strength is attributed to its ensemble nature and its ability to handle non-linear relationships and the data's skewed distribution.

Additionally, the Random Forest model's feature importance analysis highlighted User_Count and Critic_Score as the most influential factors in predicting global sales. The selection of the XG Boost Regressor confirms the project's potential to provide accurate and reliable sales forecasts for the video game industry.

2. Introduction

2.1 Background

The video game industry has evolved into one of the most dynamic and lucrative entertainment sectors worldwide. With millions of titles released across platforms ranging from home consoles, PCs to mobile devices, the competition is fierce. Sales performance often determines the long-term success or failure of a game, influencing future development budgets, marketing strategies, and publisher decisions.

Predicting the sales of a video game is highly complex because it depends on multiple factors such as genre, platform, release year, publisher reputation, critic reviews, and user ratings. Traditional market research methods may fall short when attempting to forecast success in such a multifaceted industry. This is where machine learning (ML) proves valuable. ML models can analyze large historical datasets and uncover hidden patterns that correlate game features with sales outcomes.

By applying machine learning algorithms to historical video game sales data, this project seeks to create predictive models that can provide publishers, developers, and retailers with actionable insights to guide decision-making and reduce business risks.

2.2 Motivation

There are several motivations behind undertaking this project:

- **Business Benefits:** Publishers and distributors can make better-informed decisions about production, marketing, and platform targeting by understanding which factors drive higher sales.

- **Consumer Insights:** Analysis of game attributes such as genre, rating, and critic reviews can reveal correlations with global sales and player preferences, which can help game developers design more engaging games while improving the sales.

- **Educational Value:** From a student and researcher's perspective, this project provides hands-on exposure to applying regression algorithms, feature engineering, and hyperparameter tuning in a real-world context.

- **Practical Applications of ML:** Sales forecasting is an essential use case of machine learning in business analytics. By solving this problem, learners gain practical skills in data preprocessing, model implementation, and evaluation.

3. Project Design

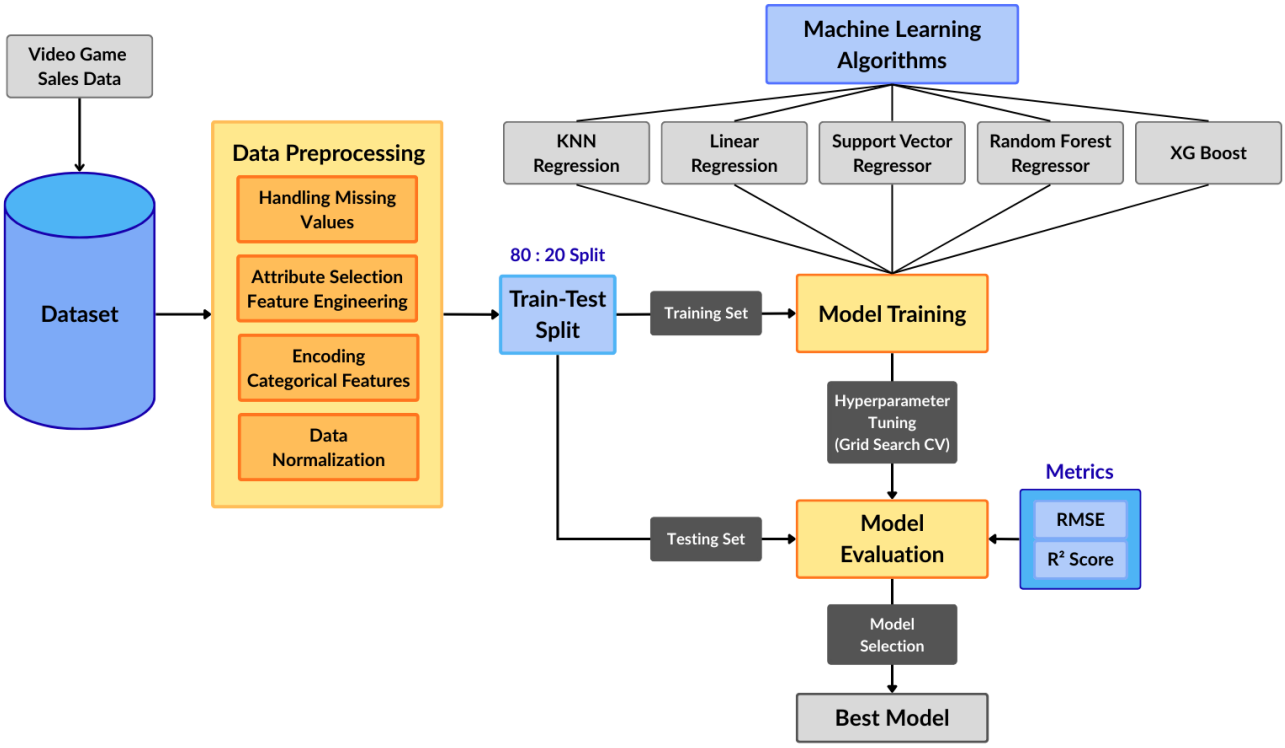


Figure - Proposed Project Design

The above flowchart explains the proposed workflow. It has the following steps:

1. **Data Collection (Video Game Sales Data):** The dataset containing video game attributes such as platform, genre, publisher, critic score, and user score is collected.
2. **Data Preprocessing:** Missing values are handled, irrelevant attributes are dropped, and categorical features are encoded. Normalization is applied to numerical features to ensure uniform scaling.
3. **Model Training:** Multiple regression algorithms are trained on the dataset to capture different relationships.
4. **Hyperparameter Tuning:** Grid Search with cross-validation systematically tests combinations of model parameters. This ensures that each algorithm performs optimally on the dataset.
5. **Model Evaluation:** Metrics such as RMSE and R^2 score are used to assess the performance of the model.
6. **Model Selection:** Based on evaluation results, the model with the lowest RMSE and highest R^2 is selected. This model represents the most accurate and reliable predictor of video game sales.

4. Dataset Overview

This dataset provides a structured representation of video game sales and associated metadata, scraped from vgchartz.com, resulting in a total of 16,598 records. Each record corresponds to one video game title and contains attributes that describe its commercial, critical, and regional performance.

4.1 Feature/Attribute Details

- **Name** - The title of the video game (e.g., *Wii Sports*, *Grand Theft Auto V*).
- **Platform** indicates the hardware system on which the game was released (e.g., PS4, Xbox 360, Wii, PC, DS, Switch).
- **Year** denotes the **year of release** of the video game. This attribute helps capture temporal trends, such as spikes in certain years due to console generations or economic conditions.
- **Genre** refers to the **category of gameplay**, such as Action, Sports, Role-Playing (RPG), Shooter, Adventure, etc.
- **Global_Sales** - The total worldwide sales of a video game, in millions. This is the target variable for prediction. It is the sum of NA, EU, JP, and Other sales.
- **Rating** - The age/content rating of the game, typically provided by boards like ESRB

  (e.g., E = Everyone, T = Teen, M = Mature).
- **Critic_Score** - The average score assigned by professional critics on a scale of 0–100. Higher critic scores generally indicate better quality and may influence consumer purchase decisions.
- **Critic_Count** - The number of critics who reviewed the game. A higher count indicates broader coverage, while a lower count may suggest limited availability or niche interest.
- **User_Score** - The average rating given by players, often on a scale of 0–10. Provides a direct measure of consumer satisfaction, which can differ from critic opinions.
- **User_Count** - The number of users who submitted ratings for the game. A high user count may reflect greater popularity or a larger player base.

4.2 Data Characteristics

- The dataset contains a mix of numerical and categorical features.
- Some columns (e.g., *Year, Rating, Critic_Score, User_Score*) have missing or inconsistent values that must be addressed during preprocessing.
- The sales data are skewed, with a small number of blockbuster games dominating global sales, while most games sell at relatively modest levels.

5. Data Preprocessing

Raw datasets often contain inconsistencies, missing values, and attributes that are not directly suitable for machine learning models. Therefore, preprocessing is a crucial step in preparing the data for predictive modelling. In this project, preprocessing involved four major tasks: handling missing values, attribute selection and feature engineering, encoding categorical variables, and data normalization.

5.1 Handling Missing Values

The dataset includes several features such as *Year*, *Publisher*, *Rating*, *Critic_Score*, and *User_Score* that contain missing values. If left untreated, these gaps can negatively affect the performance of regression models. Different strategies were employed depending on the nature of the attribute:

- **Removal of Irrelevant Records:** Records with extensive missing information across multiple key fields were dropped from the dataset.

- **Imputation:** For numerical features such as *Critic_Score* or *User_Score*, missing values were replaced with the mean of available values. For categorical features such as *Publisher* or *Rating*, missing entries were filled with the mode (most frequent value).

- This ensured that the dataset remained consistent and avoided losing valuable information due to excessive record deletion.

5.2 Attribute Selection and Feature Engineering

Not all available attributes contribute equally to predicting global sales. Attributes such as *Name* and *Rank, Publisher* were excluded from modeling, since they do not carry predictive value. Instead, the following features were retained: *Platform, Year, Genre, , Critic_Score, Critic_Count, User_Score, User_Count, Rating,* and the four regional sales figures.
Additionally, feature engineering was performed:

- The four regional sales variables (*NA_Sales, EU_Sales, JP_Sales, Other_Sales*) were summed to verify consistency with the provided *Global_Sales*.
- The Age of the Game is derived from *Year* to capture sales trends over different console generations.

5.3 Encoding Categorical Variables

Since most machine learning regression algorithms require numerical input, categorical variables had to be encoded:

Label Encoding is applied to ordinal features such as *Rating, Platform, Genre*, where values have a meaningful order (e.g., “E” < “T” < “M” for Rating).

Encoding ensured that categorical information was preserved and made suitable for model training.

5.4 Data Normalization

The dataset contains attributes with varying scales, such as critic scores (0–100), user scores (0–10), and sales figures (measured in millions). Without normalization, models like K-Nearest Neighbors (KNN) and Support Vector Regression (SVR) would give undue importance to attributes with larger numerical ranges.

- To address this, **Z-Score Normalization** was applied to numerical columns. It is a scaling technique that transforms the data so that it has a **mean of 0 and a standard deviation of 1**. Each value is adjusted based on how far it deviates from the mean, relative to the standard deviation:

$$z = \frac{x - \mu}{\sigma}$$

  Where:

  - x = original data value
  - μ = mean of the feature
  - σ = standard deviation of the feature

- The **log1p transformation** is a logarithmic transformation applied to skewed numerical data(the global sales feature in this case), defined as:

$$y = \log(1 + x)$$

  The "+1" ensures that zero values can also be transformed without errors.

  This technique is particularly useful for **highly skewed distributions**, such as video game sales, where a small number of blockbuster games sell extremely well while the majority sell modestly.

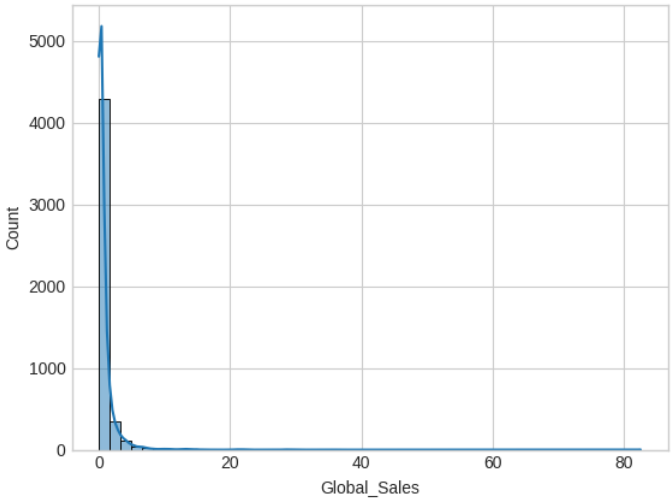


Figure - Before Log Transformation

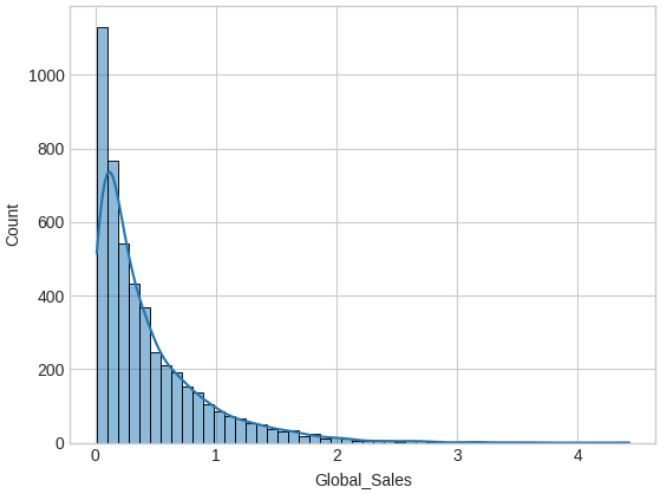


Figure - After Log Transformation

6. Model Training and Hyperparameter Tuning

After preprocessing, the dataset was split into training and testing sets in an 80: 20 ratio. The target variable was the log-transformed `Global-Sales`. Multiple regression algorithms were applied to capture different predictive relationships:

1. K-Nearest Neighbour Regressor
2. Linear Regression
3. Support Vector Regressor
4. Random Forest Regressor

Training Strategy Used:

- **Train-Test Split:** (80 : 20 Ratio) 3876 training samples, 969 testing samples.
- **Cross-Validation:** 5-fold cross-validation was used for hyperparameter optimization.
- **Grid Search:** Applied for SVR and Random Forest to identify the best set of parameters.

6.1 K-Nearest Neighbour Regressor

KNN is a **non-parametric algorithm**. Predictions are based on the average target value of the Nearest-K Neighbours in feature space.

$$\hat{y} = \frac{1}{K} \sum_{i \epsilon N_k(x)} y_i$$

Where:

- $N\_k\ (x)$ – Set of K Neighbours of x
- y_i – Observed Sales of Neighbour i

Hyperparameter: Number of Neighbours (K)

- Small K – Sensitive to outliers
- Large K – Over smoothing

The K-value is tuned using 5-fold cross validation and the optimal value is found to be $K = 9$

```
from sklearn.neighbors import KNeighborsRegressor

k_values = range(1, 31)
cv_scores = []
for k in k_values:
  knn = KNeighborsRegressor(n_neighbors=k)
  scores = cross_val_score(knn, X_train, y_train, cv=5, scoring='r2')
  cv_scores.append(scores.mean())

optimal_k = k_values[np.argmax(cv_scores)]
print('Optimal K-Neighbors = ', optimal_k)
```

```
Optimal K-Neighbors =  9
```

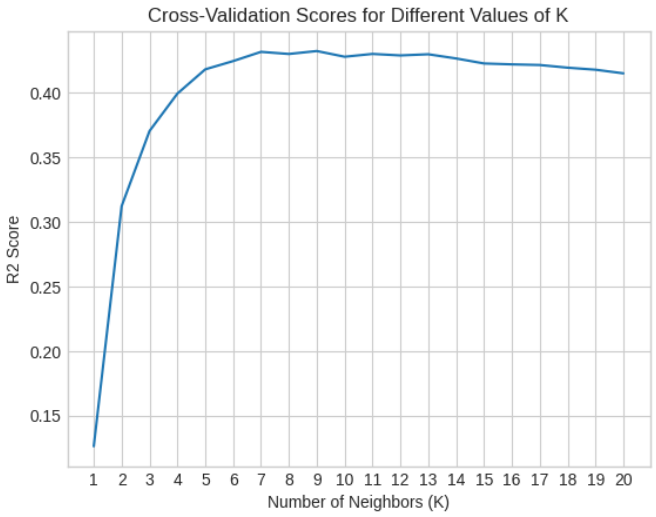


Figure - R^2 Scores Plot for Different Values of K

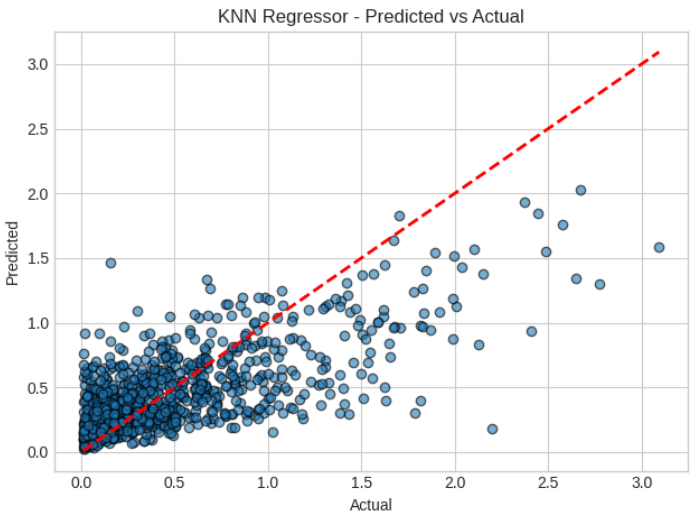


Figure - KNN Regressor Predicted vs. Actual Values Plot

6.2 Linear Regression

Linear regression assumes a **linear relationship** between input features and the target variable.

Mathematical model:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

where:

- $\hat{y}$ – Predicted Global Sales
- x_i – Input feature
- β_i – Learned Coefficients
- β_0 – Intercept

Linear regression is simple and interpretable but struggles when relationships are non-linear or when multicollinearity exists.

```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)
```

```
LinearRegression
LinearRegression()
```

```
y_pred = lin_reg.predict(X_test)
rmse = root_mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print('Root Mean Squared Error: ', rmse)
print('R2 Score: ', r2)
```

```
Root Mean Squared Error:  0.38169078603888235
R2 Score:  0.39248875031902164
```

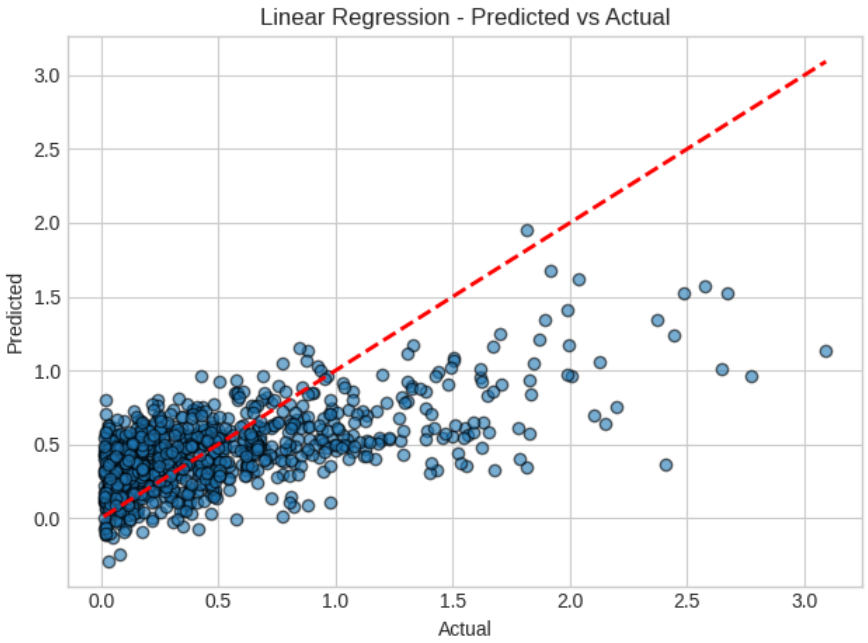


Figure - Linear Regression Predicted vs. Actual Values Plot

6.3 Support Vector Regressor

SVR extends Support Vector Machines (SVM) to regression. It aims to fit a function within an **ε-insensitive margin**.

Objective: $$f(x) = w^T x + b$$

Loss function:

$$L_\epsilon(y, f(x)) = \begin{cases} 0 & , \quad \text{if } |y - f(x)| \leq \epsilon \\ |y - f(x)| - \epsilon, & \quad \text{Otherwise} \end{cases}$$

Hyperparameters:

- **C (regularization):** Trade-off between model complexity and margin violations.
- **γ (gamma):** Defines influence of single training samples in RBF kernel.
- **ε (epsilon):** Margin of tolerance for errors.

SVR effectively captures non-linear relationships but is computationally intensive.

```
from sklearn.svm import SVR

param_grid = {'C': [0.01, 0.1, 1, 10],
              'gamma': [0.0001, 0.001, 0.01, 0.1, 1],
              'kernel': ['rbf']}

grid = GridSearchCV(SVR(), param_grid, cv=5, scoring='r2')
grid.fit(X_train, y_train)

svr = grid.best_estimator_

print('Best Parameters: ', grid.best_params_)
```

```
Best Parameters:  {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
```

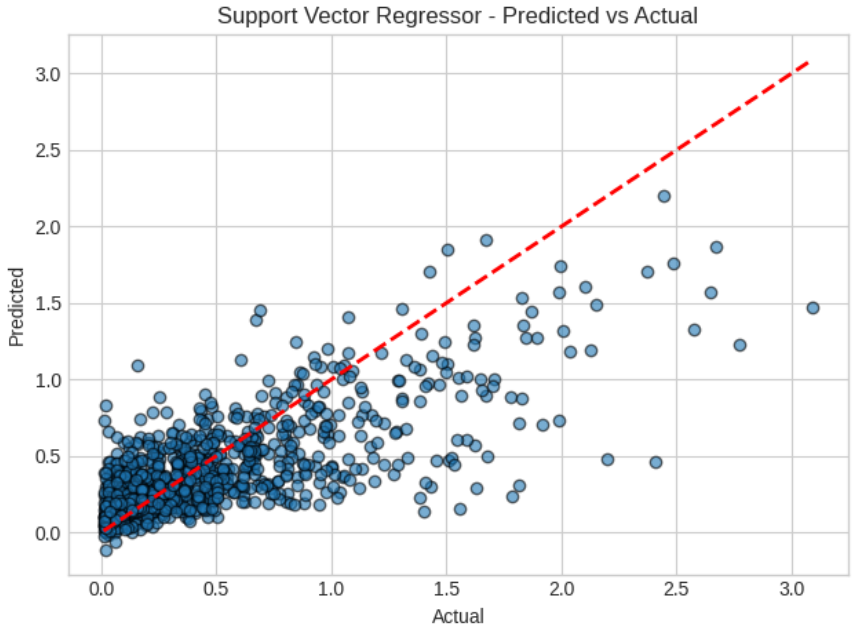


Figure – Support Vector Regressor Predicted vs. Actual Values Plot

6.4 Random Forest Regressor

Random Forest is an **ensemble learning method** that combines multiple decision trees. Each tree is trained on a random subset of data and features, reducing overfitting and variance.

$$\hat{y} = \frac{1}{T}\sum_{t=1}^{T} h_t(x)$$

Where:

- T – Number of Trees
- $h_t(x)$ – Prediction from tree t

Key hyperparameters:

- **n_estimators:** Number of trees.
- **max_depth:** Maximum depth of each tree.
- **min_samples_split:** Minimum samples required to split a node.

Feature Importances in Random Forest:

Feature importance in Random Forest tells how much each feature contributes to reducing prediction error across the ensemble of decision trees. It is a way to measure which variables are the most influential in determining the target (here, global video game sales).

Higher importance value = feature contributes more to prediction.

Feature importance scores are relative; they sum to 1 across all features.

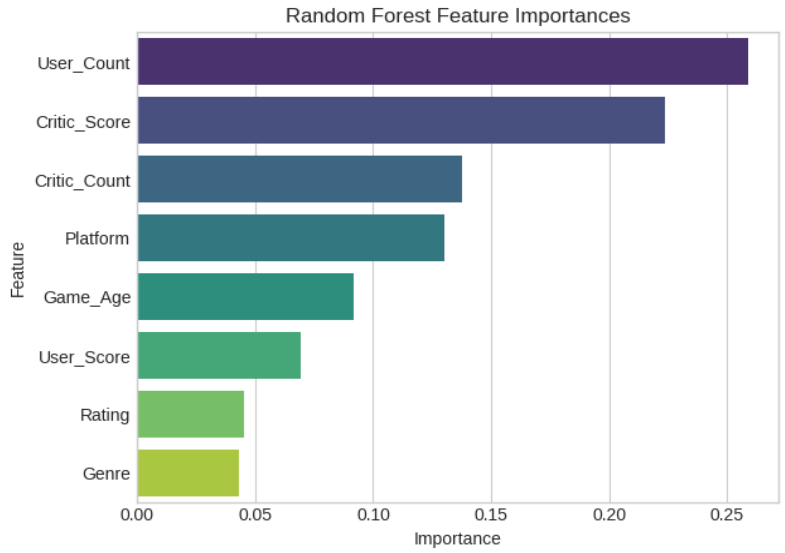


Figure - Feature Importances

From the above plot, we can observe that **User_Count, Critic_Score** are the top most features that contributed in reducing the prediction error across the ensemble of decision trees.

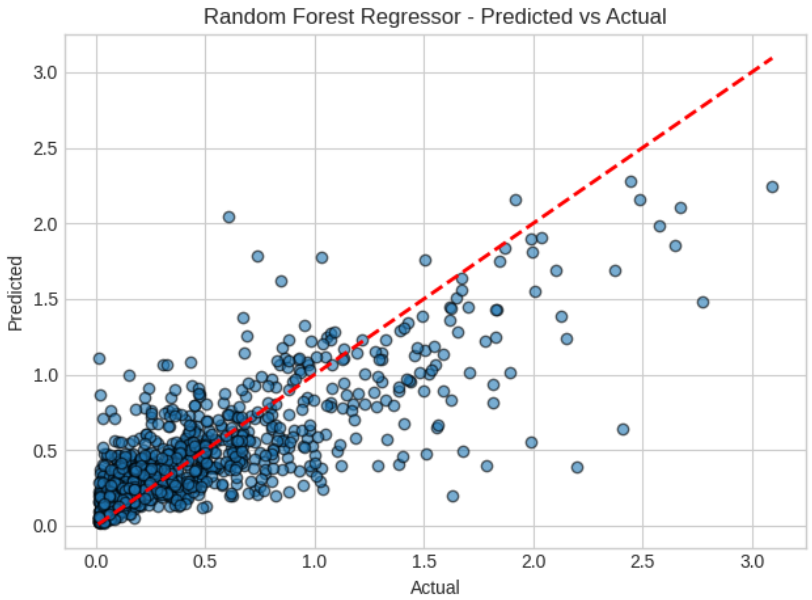


Figure – Random Forest Regressor Predicted vs. Actual Values Plot

```
param_grid = {'n_estimators': [100, 200, 300],
              'max_depth': [None, 10, 20],
              'min_samples_split': [2, 5, 10]}

grid = GridSearchCV(RandomForestRegressor(random_state=42),
                    param_grid, cv=5, scoring='r2', n_jobs=-1)
grid.fit(X_train, y_train)

rf_reg = grid.best_estimator_

print('Best Parameters: ', grid.best_params_)
```

```
Best Parameters:  {'max_depth': 20, 'min_samples_split': 2, 'n_estimators': 300}
```

6.5 XG Boost Regression

XG Boost (eXtreme Gradient Boosting) is a powerful and popular gradient boosting framework that uses decision trees as base estimators. It's an **ensemble method** known for its speed and performance, often winning machine learning competitions.

1. It builds trees sequentially, where each new tree tries to **c**orrect the prediction errors (residuals) of the previous ensemble tree.
2. It uses a gradient descent approach to minimize the loss function (the difference between the actual and predicted sales) when adding new trees.
3. It includes regularization terms (L1 and L2) in the objective function to control the complexity of the model, which helps prevent overfitting and improves the model's generalization ability.
4. The final prediction for a given video game is the sum of the predictions from all the individual decision trees.

$$\hat{y} = \sum_{k=1}^{K} f_k(x)$$

Where:

- $\hat{y}$ is the predicted log sales for input x
- K is the total number of trees.
- $f_k(x)$ is the prediction of the k-th tree for input x

Key Hyperparameters:

- **N_estimators:** The number of boosting rounds (number of trees to build).
- **Max_depth:** Maximum depth of a tree.
- **Learning_rate:** Shrinkage of the feature weights after each boosting step.

```
xgb_model.fit(X_train, y_train)

# Make predictions
y_pred = xgb_model.predict(X_test)

# Evaluate the model
y_pred = xgb_model.predict(X_test)
rmse = root_mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Root Mean Squared Error: {rmse:.4f}")
print(f"R2 Score: {r2:.4f}")
```

```
Root Mean Squared Error: 0.2885
R2 Score: 0.6528
```

7. Model Evaluation and Selection

After training multiple regression models and tuning their hyperparameters, model evaluation and model selection are the final, critical steps in the machine learning workflow. This process involves assessing how well each model performs on unseen data and then choosing the most accurate and reliable one for predicting video game sales.

7.1 Model Evaluation

Model evaluation is the process of quantitatively assessing a trained machine learning model's performance on the testing dataset (unseen data). It measures the model's ability to generalize and make accurate predictions in a real-world context.

Evaluation Metrics Used

1. **R^2 Score (Coefficient of Determination)**
   The R^2 score is a goodness-of-fit measure that indicates the proportion of the variance in the dependent variable that is predictable from the independent variables. In essence, it shows how well the model's predictions approximate the actual sales data.
   A score of 1.0 indicates a perfect fit, while a score of 0.0 suggests the model is no better than simply predicting the mean of the target variable.

   The formula for the R2 score is:

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

   Where:

   - y_i is the actual value of Global Sales.
   - $\hat{y}_i$ is the predicted value of Global Sales.
   - $\bar{y}$ is the mean of the actual Global Sales values.

2. **Root Mean Squared Error (RMSE)**
   RMSE is the square root of the average of the squared differences between predicted sales and actual sales. It is a measure of the average magnitude of the error. Because the errors are squared before being averaged, RMSE gives a higher weight to large errors, making it a good metric to penalize significant prediction mistakes.
   The units of RMSE are the same as the target variable (log-transformed sales), making it easy to interpret. A lower RMSE indicates better model performance.

   The formula for RMSE is:

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

	Model	R2_Score	RMSE
0	XG Boost	0.652830	0.288540
1	Random Forest Regressor	0.626111	0.299437
2	Support Vector Regressor	0.529424	0.335930
3	KNN Regression	0.500039	0.346260
4	Linear Regression	0.392489	0.381691

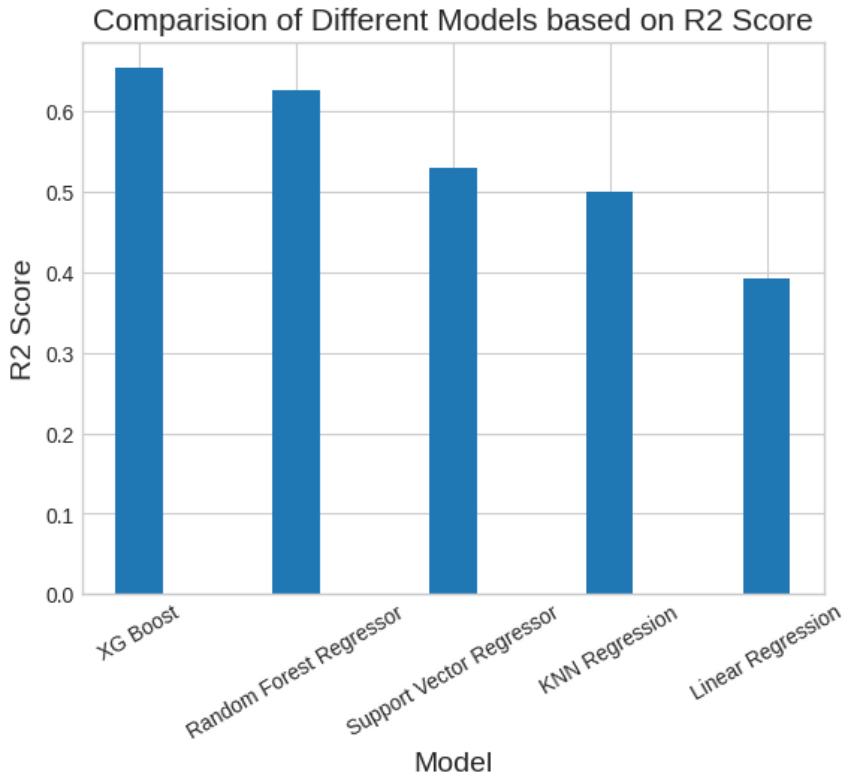
7.2 Model Selection

The **XG Boost Regressor** is selected as the final model. This selection is based directly on the model evaluation results, which prove its superior predictive potential.

- **Highest R^2 Score:** XG Boost achieved the highest score of **0.6528**, indicating that it explains approximately **65.3%** of the variance in the log-transformed global sales, making it the most accurate model.

- **Lowest RMSE:** It also recorded the lowest RMSE **(0.2885)**, signifying that its predictions have the smallest average magnitude of error compared to the other models.

The strength of the XG Boost Regressor in this task is attributed to its ensemble nature and its ability to effectively handle the non-linear relationships and skewed distribution present in the video game sales data. Its strong performance positions it as a reliable tool for forecasting future video game sales.



8. Conclusion

This machine learning project successfully developed a robust framework for predicting global video game sales, offering valuable foresight into one of the world's most dynamic and competitive entertainment sectors. The primary goal was to create predictive models that could provide publishers, developers, and retailers with actionable insights to guide strategic decisions, thereby reducing the significant business risks associated with new title releases.

The foundation of our success lay in meticulously preparing the raw data. This involved crucial Data Preprocessing steps, such as handling missing values through imputation and applying transformations like Z-Score Normalization to numerical features for uniform scaling. Most importantly, the target variable, Global_Sales, was subjected to a log transformation to address its high skewness, which significantly improved the models' ability to accurately predict sales across the entire range, including the vast majority of games with modest figures.

Summary of Results and Benefits

By training and evaluating five different regression algorithms—including Linear Regression, Random Forest, and XG Boost—the project established a clear best-in-class model. The XG Boost Regressor was selected as the final, most reliable predictor due to its superior performance metrics, achieving the lowest Root Mean Squared Error (RMSE) and the highest R^2 Score (0.6528). This means the model can explain nearly two-thirds of the variation in video game sales, a powerful capability for forecasting.

The analysis also yielded important consumer insights, particularly from the Random Forest model's feature importance results. These results identified User_Count and Critic_Score as the most influential factors driving global sales. This insight directly informs developers and publishers, confirming that consumer satisfaction (User_Count) and critical acclaim are paramount for commercial success.

Future Scope

Looking ahead, the project offers several pathways for further development. The model's predictive power could be enhanced by integrating more complex, non-structured data, such as sentiment analysis from social media reviews or pre-release hype metrics. Additionally, exploring deep learning models, such as Recurrent Neural Networks (RNNs) or Long Short-Term Memory (LSTM) networks, could potentially capture more intricate temporal dependencies and complex non-linear relationships than traditional ensemble methods. Finally, the framework could be extended to predict regional sales independently, allowing for highly specific marketing and distribution strategies tailored to individual markets like North America, Europe, or Japan

9. References

Project GitHub Repository Links

Dataset Link from Kaggle Website

https://www.kaggle.com/datasets/gregorut/videogamesales

Scikit-Learn Documentation

https://scikit-learn.org/stable/